

10 APACHE SPARK CONCEPTS EVERY DATA ENGINEER MUST KNOW



BY MICHAEL FERNANDES

1.RESILIENT DISTRIBUTED DATASET (RDD)

RDDs (Resilient Distributed Datasets) are Spark's original distributed data structure that stores data across multiple machines in a cluster

RDDs introduced the core concepts of distributed computing and fault tolerance that Spark still uses today.

Use Cases

- Complex transformations
- Low-level data processing
- Custom distributed algorithms



2. DATAFRAMES

A DataFrame is a distributed collection of data organized into rows and columns, similar to a SQL table.

Most modern Spark applications are built using DataFrames because they offer simplicity and high performance.

Ways to Create DataFrame in Spark

Hive Data

Csv Data

Json Data

RDBMS Data

XML Data

Parquet Data

Cassandra Data

RDDs

Spark SQL

DataFrame

	Col1	Col2	Col3	
Row 1				
Row 2				
Row 3				
.				
.				

3. TRANSFORMATIONS VS ACTIONS

Transformations create a new DataFrame or RDD without executing immediately.

Actions trigger execution and return results.

Transformations

- > **map()** – Apply function
- > **flatMap()** – Split + flatten
- > **filter()** – Keep matching rows
- > **distinct()** – Remove duplicates
- > **union()** – Merge datasets
- > **join()** – SQL-style joins
- > **select()** – Pick columns
- > **withColumn()** – Add/modify column
- > **orderBy()** – Aggregate

Actions

- > **collect()** – Return all rows
- > **show()** – Display sample rows
- > **count()** – Count rows
- > **first()** – First row
- > **take(n)** – Return first n rows
- > **saveAsTextFile()** – Save ROD
- > **write.parquet()** – Save DataFrame
- > **foreach()** – Apply function per row

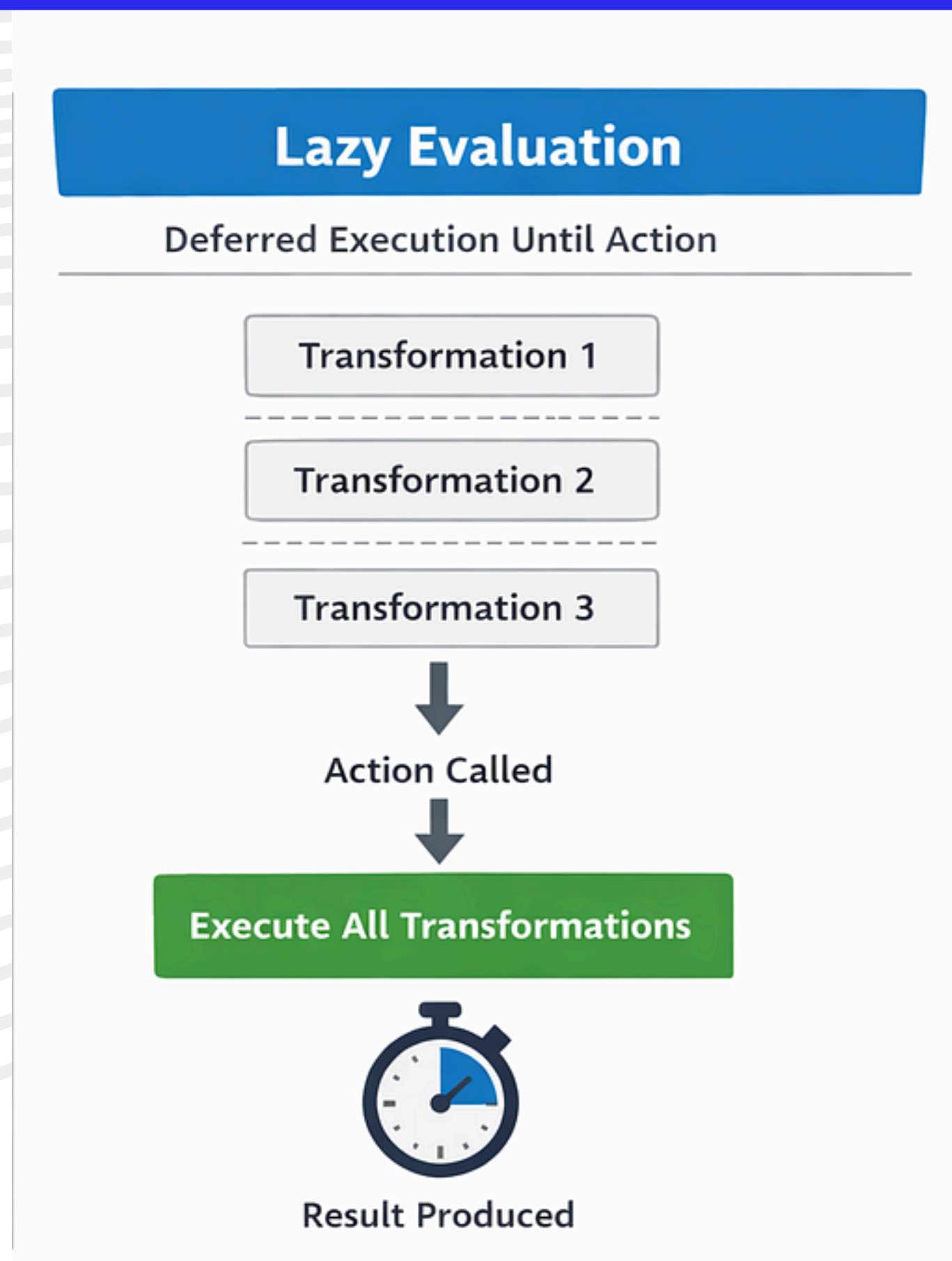
4. LAZY EVALUATION

Spark does not execute transformations as they are written.

Instead, it records them and creates an execution plan.

You can chain multiple transformations together, but Spark waits until an action is triggered before processing any data.

Lazy evaluation is one of the primary reasons Spark achieves high performance at scale.



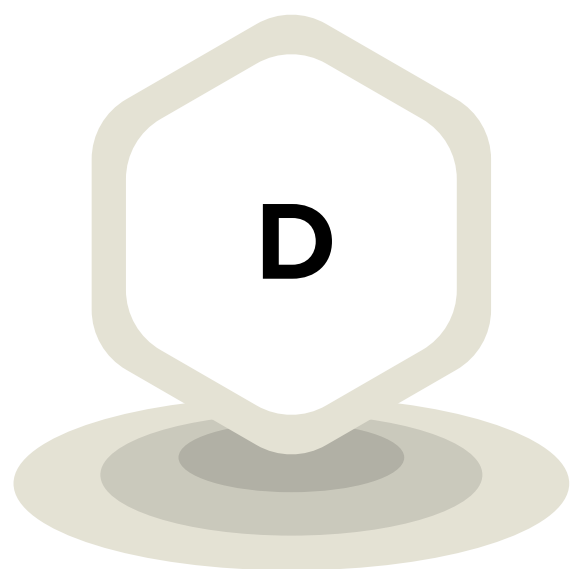
5.DAG (DIRECTED ACYCLIC GRAPH)

A DAG (Directed Acyclic Graph) is Spark's execution plan that represents the sequence of transformations applied to data.

Spark builds the DAG before executing the application.

The DAG allows Spark to:

- Optimize execution
- Improve parallel processing
- Reduce unnecessary computation
- Efficiently schedule tasks



DIRECTED

Operations follow
a specific direction



ACYCLIC

No circular
dependencies
exist



GRAPH

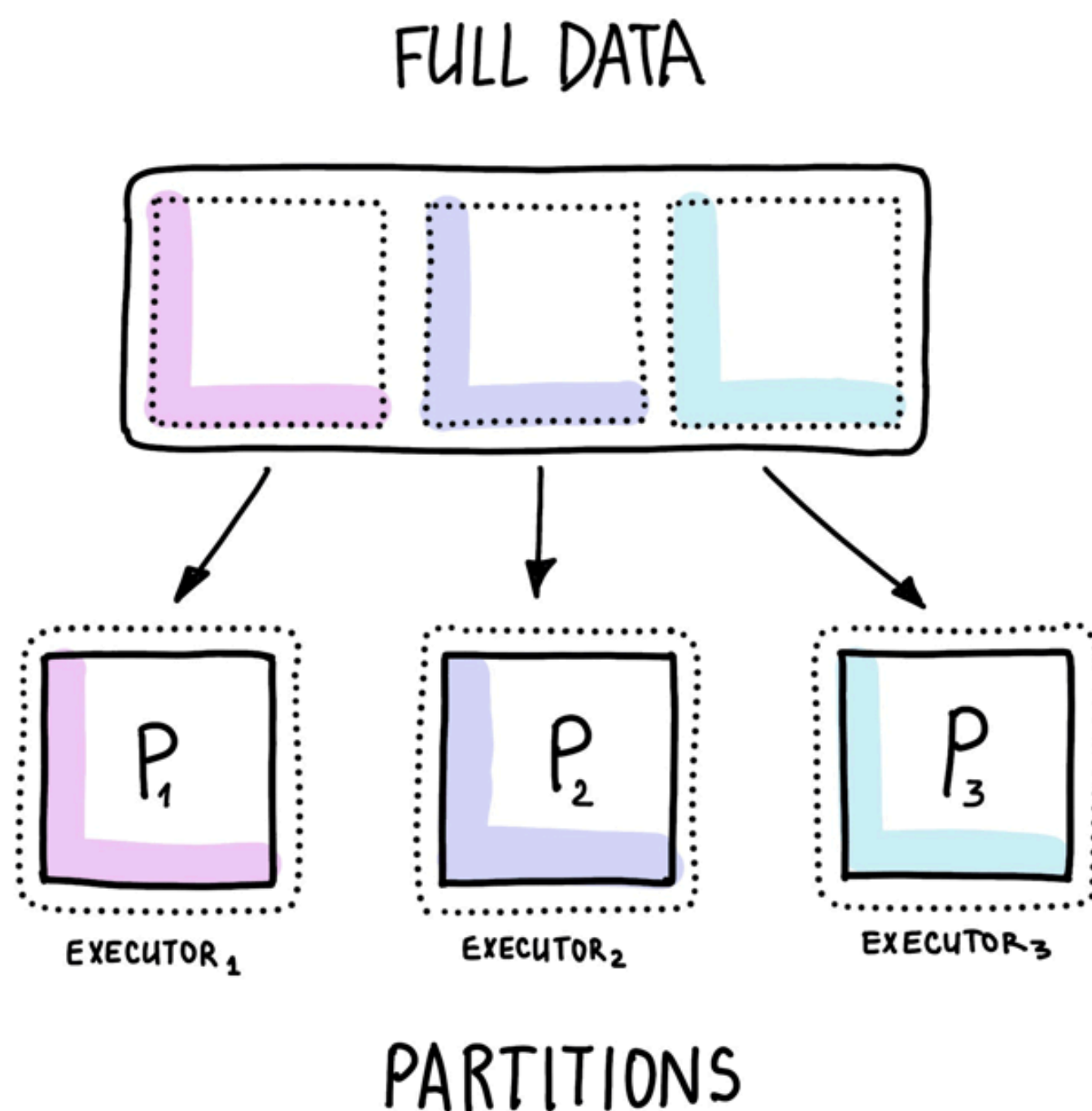
Represents workflow
as connected
operations

6. PARTITIONS

Partitions are logical chunks of data distributed across worker nodes.

Each partition can be processed independently.

- Parallel processing
- Scalability
- Load distribution
- Improved performance



@luminousmen.com

7.CACHING & PERSISTENCE

By default, Spark recomputes transformations every time an action is triggered. This can become expensive for:

- Iterative algorithms
- Machine Learning pipelines
- Repeated queries
- Interactive analytics

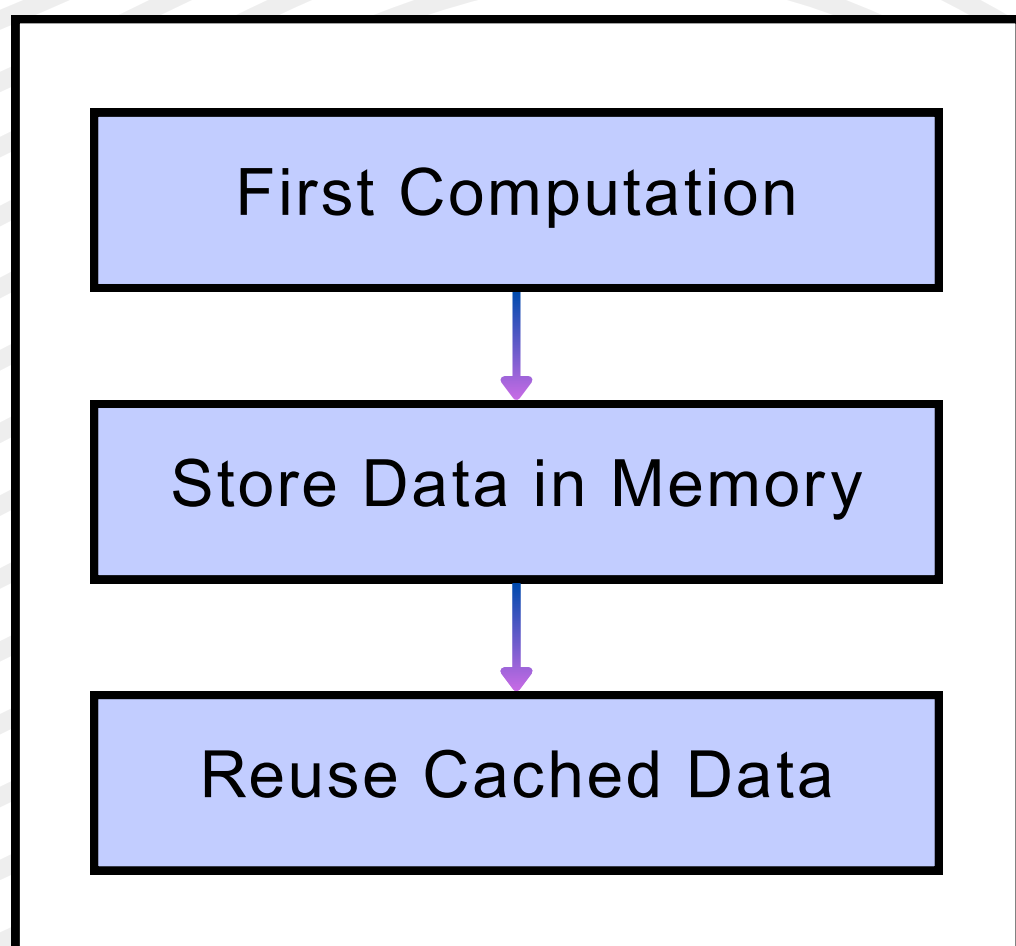
Caching stores intermediate data so Spark can reuse it directly.

`persist()` allows Spark to store datasets using customizable storage levels. It is useful for:

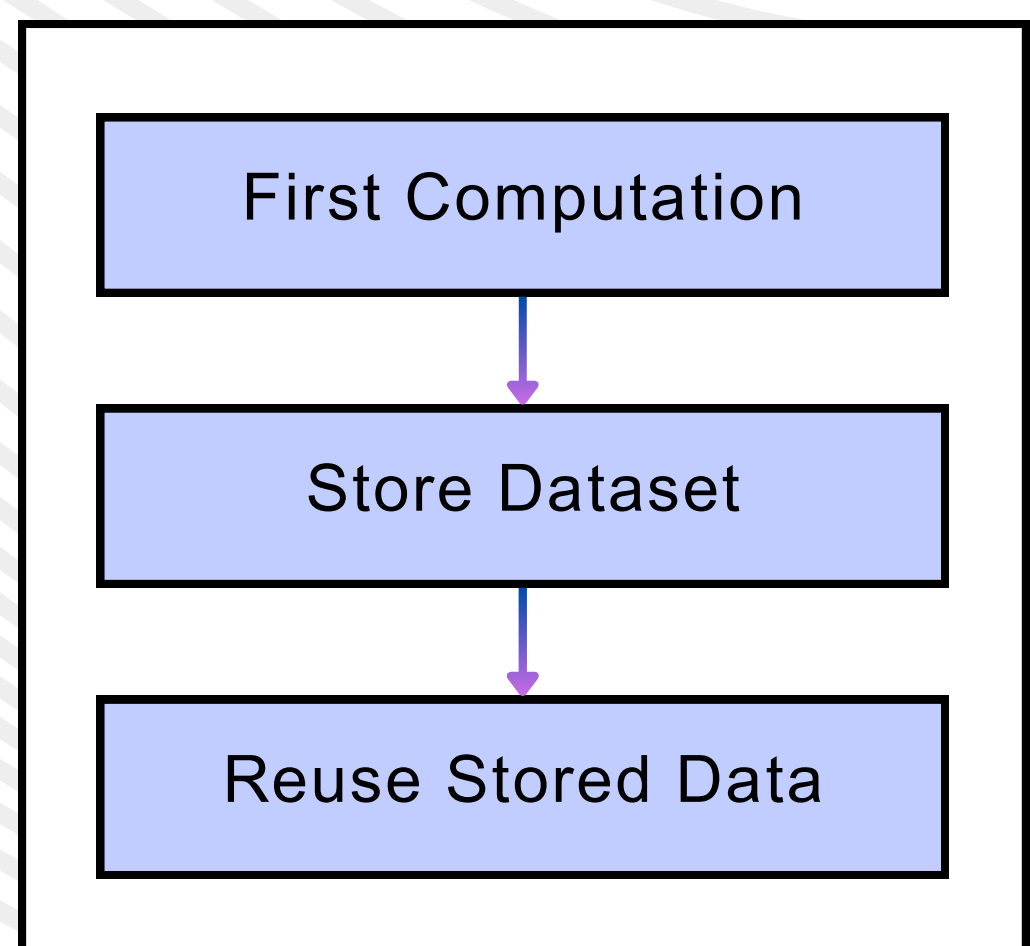
- Large datasets
- Iterative workloads
- Memory optimization
- Repeated computations

Unlike `cache()`, it can spill data to disk when memory is insufficient.

Syntax : `df.persist()`



`cache()`

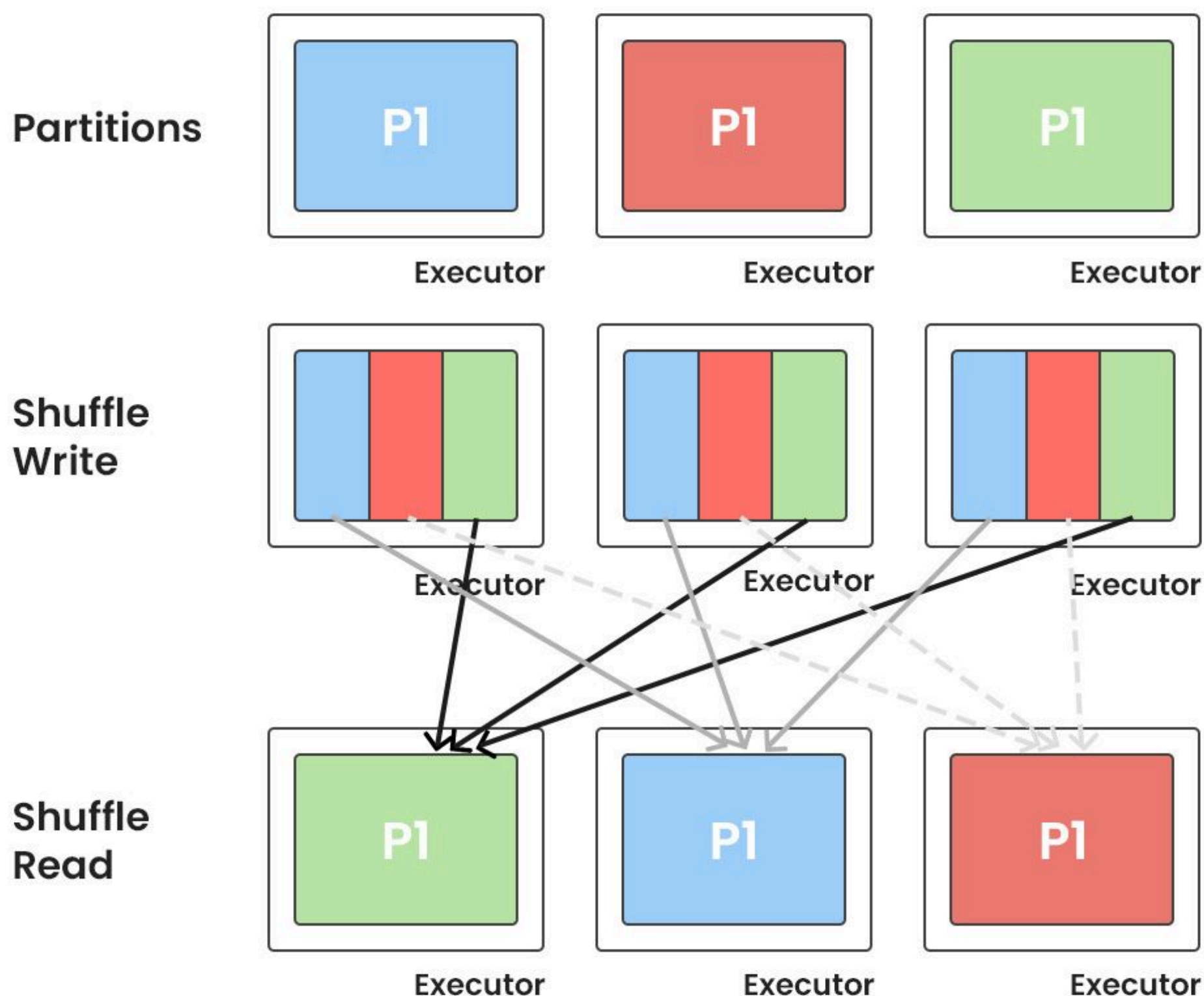


`persist()`

8.SHUFFLE OPERATIONS

A shuffle in Apache Spark is the process of redistributing and reorganizing data across different partitions and executor nodes so that related records are co-located on the same physical machine.

This mechanism is triggered by "wide transformations" where Spark cannot complete the operation using only the data already available in a single input partition



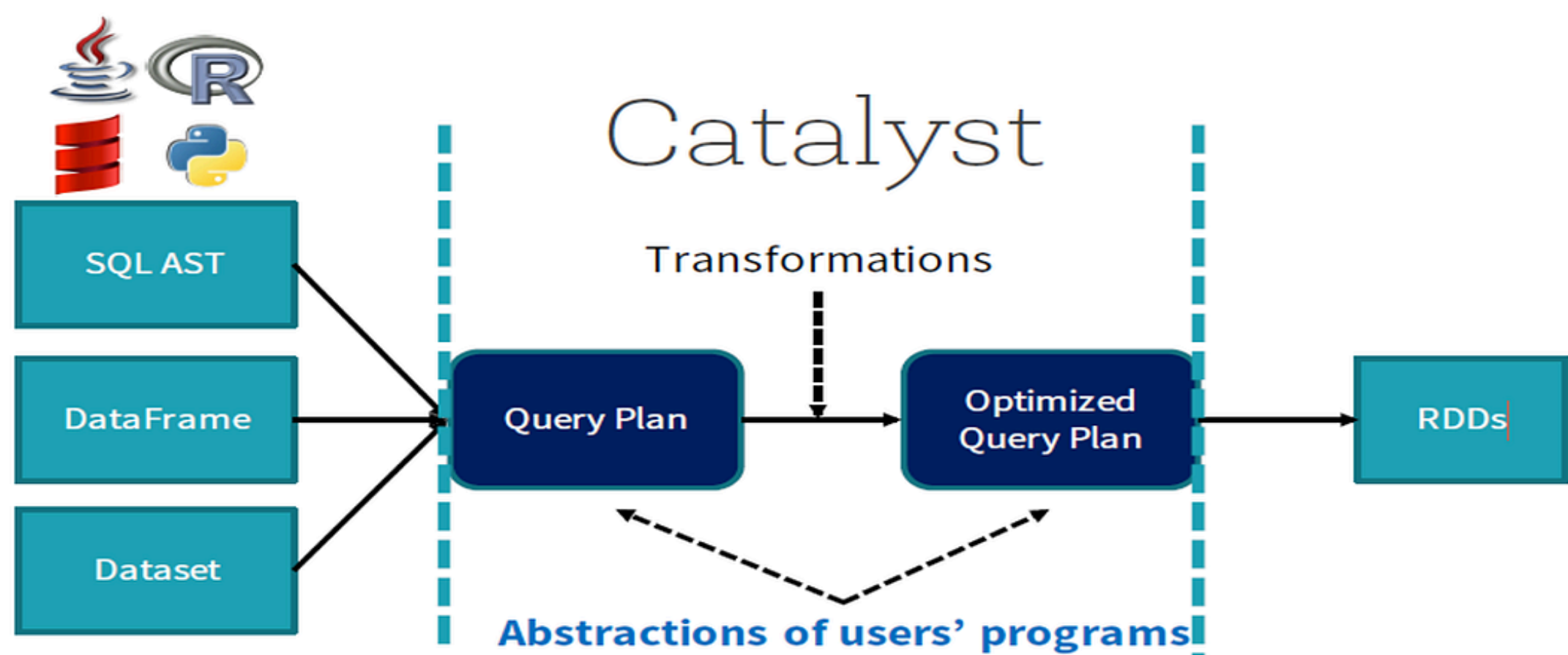
9. CATALYST OPTIMIZER

The Catalyst Optimizer is the core query optimization engine inside Apache Spark SQL.

It automatically translates high-level SQL queries and DataFrame transformations into highly efficient execution plans,

the optimization process operates on tree structures and is divided into four main phases:

- 1. Analysis: Takes the user's unresolved logical plan and queries the Catalog to resolve column names, table schemas, and data types.
- 2. Logical Plan Optimization: Uses rule-based transformations to apply logical optimizations
- 3. Physical Planning: Translates the optimized logical plan into multiple physical execution strategies and uses a cost-based model to select the best one.
- 4. Code Generation: Compiles the selected physical plan into optimized Java bytecode for execution



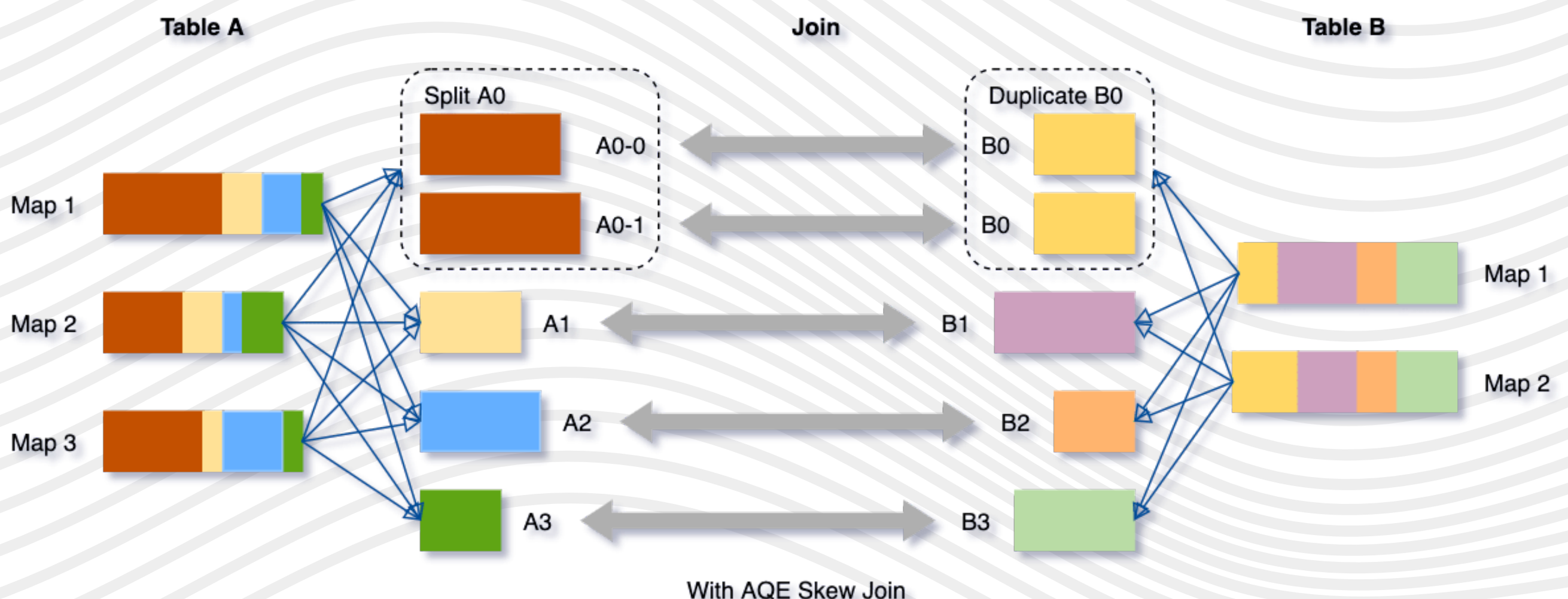
10. ADAPTIVE QUERY EXECUTION (AQE)

Adaptive Query Execution (AQE) is an optimization framework in Apache Spark.

It dynamically re-optimizes execution plans mid-query using the most accurate runtime statistics. Instead of relying solely on pre-computed (and often stale) statistics, AQE evaluates the data at materialization points (after shuffles or broadcasts) to make smarter execution choices

Key Features:

1. Dynamically Coalescing Shuffle Partitions
2. Dynamically Switching Join Strategies
3. Dynamically Optimizing Skew Joins



The background of the image consists of numerous thin, light gray wavy lines that flow across the frame from left to right, creating a sense of movement and depth. These lines are of varying frequencies and amplitudes, giving the background a textured, organic feel.

THANK YOU